CCAI RESEARCH SEMINAR · ML × MATHEMATICS

AlphaProof

Olympiad-level formal mathematical reasoning with reinforcement learning

Thomas Hubert, Rishi Mehta, Laurent Sartran *et al.*

Nature **651**, 607–613 · 19 March 2026

DOI `10.1038/s41586-025-09833-y`

COMPANION: `JUPYTERBOOK/` · `LEAN_EXAMPLES/` · `CHUNKS/`

PART I — THE PROBLEM

The first AI silver medalist at the IMO.

How did a closed-source RL agent solve problems that 99% of human contestants could not — and what can an academic lab steal from it?

The claim in one slide

> The first AI system to reach **silver-medal** level at the International Mathematical Olympiad.

IMO 2024 score **28 / 42**

- silver = 22 pts · gold = 29 pts
- **AlphaProof** solved **3 of 5 non-geometry** problems
- **combined system** solved **4 of 6** total problems
- solved **P6** — only **5 / 609** humans did
- geometry handled by **AlphaGeometry 2**

Why it is news

- Proofs are *formal Lean 4* — kernel-verified
- No human evaluators, no reward model
- The RL loop had a **perfect reward signal**
- First time *any* AI reached IMO medal level

Talk map

Core 25-minute route

1. claim and stakes
2. Lean as an RL environment
3. proof network + PUCT search
4. auto-formalization + main RL
5. TTRL as the key new idea
6. benchmark table + IMO 2024
7. one end-to-end theorem
8. three takeaways

If time runs short

- skip the open-ecosystem block
- compress the P6 proof slide to one sentence
- keep five slides only: Lean RL env · Main RL loop · TTRL loop · Numbers · One theorem

Why formal math? The one-slide motivation

Paradigm	Verifiable?	RL signal	Scale?
LLM → LaTeX	❌	needs humans	✅
LLM → Python (code RL)	✅	unit tests	✅
LLM → Lean	✅ **kernel**	**type-check**	*formal data is scarce*

> Lean turns proof search into a **sparse-reward, episodic, discrete-action MDP with a perfect verifier.** That is exactly the regime AlphaZero was designed for.

Lambda calculus in 90 seconds

Syntax

- variable: `x`
- abstraction: `λx. t`
- application: `t u`

Computation

$$(\lambda x.\ t)\, u \to t[x := u]$$

Lean notation

```
fun x => x
fun x => fun y => x
```

Why it matters here

- implication proofs are functions
- proof terms are structured functional expressions

AlphaProof ultimately searches for typed objects of this kind.

AlphaProof in one linear pipeline

$$\text{informal problem} \to \text{Lean theorem} \to \text{tactic state} \to \text{proof network} \to \text{tree search} \to \text{verified proof}$$

1. auto-formalize informal mathematics into Lean
2. search over tactics inside the Lean kernel
3. train on proved, disproved, and timed-out attempts
4. at test time, optionally run **TTRL** on variants of the target

Speaker note

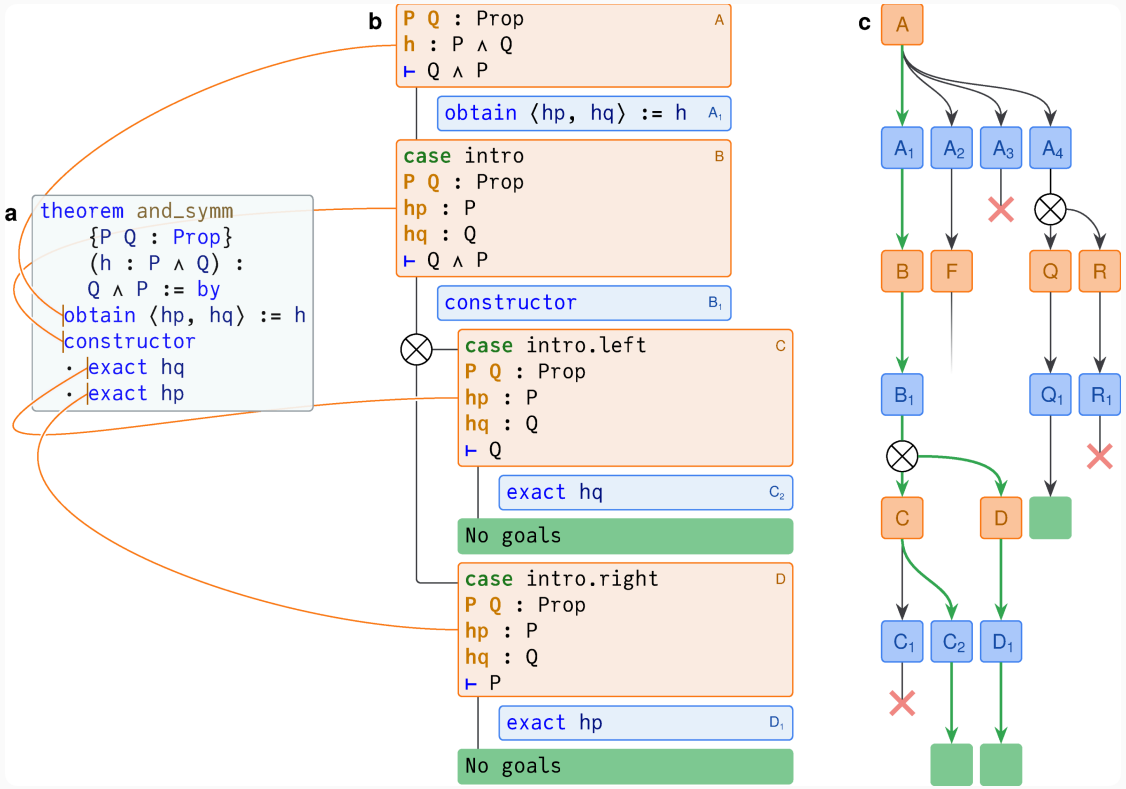
- every later slide just zooms in on one arrow of this chain
- if the audience gets lost, come back to this slide

PART II — INSIDE ALPHAPROOF

Environment · policy · search.

The three AlphaZero ingredients, re-imagined for a theorem prover.

The Lean RL environment



MDP framing

- **State** s_t — *tactic state*: hypotheses, goals, metavars
- **Action** a_t — a *tactic*, i.e. a string the Lean kernel runs
- **Reward** $r_t = -1$ for every tactic application
- **Return** $G_t = -(\text{steps left})$

Branching

- A tactic can split a goal into several subgoals (⊗)
- AND-node value = **minimum** child return
- So the value tracks the **hardest open branch**

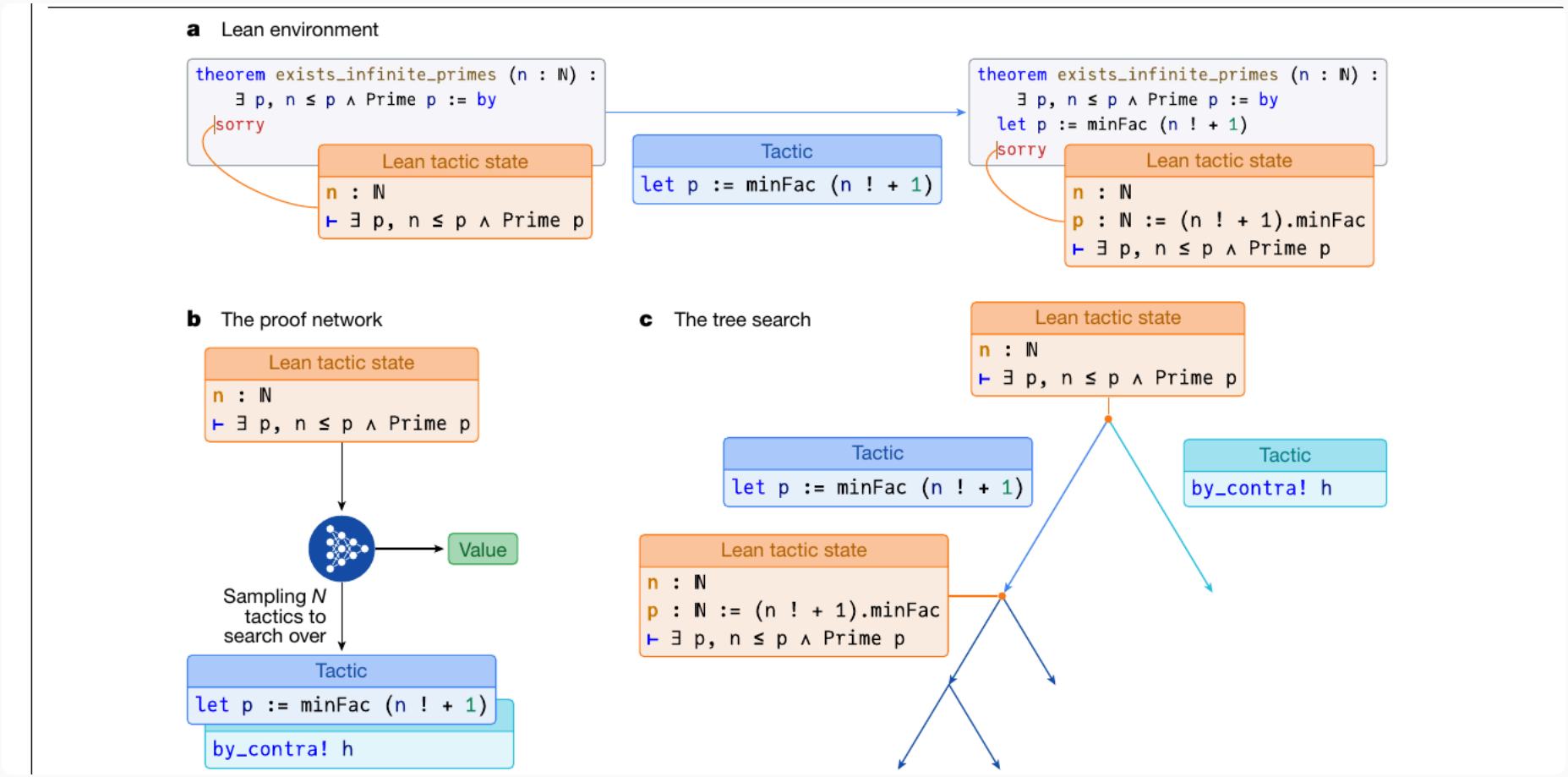
A tactic state is just text

```
theorem and_symm {P Q : Prop} (h : P ∧ Q) : Q ∧ P := by
  obtain ⟨hp, hq⟩ := h      -- a1   (destructs the conjunction)
  constructor               -- a2   (creates two subgoals — ⊗)
  · exact hq                -- a3
  · exact hp                -- a4   → No goals
```

After `obtain`, the agent literally reads the string: `P Q : Prop · hp : P · hq : Q · ⊢ Q ∧ P`

> The **state is a string**. The **action is a string**. A finished 4-step proof has return $G_0 = -4$.

AlphaProof's three components



(a) Lean env · **(b)** Proof network (policy + value) · **(c)** PUCT tree search

The proof network

Architecture

- **3 B encoder–decoder transformer**
- Input = **text-rendered tactic state** + pretty-printed top-level statement
- Sequence-to-sequence — tactics are *generated*, not picked from a list

Outputs per state s

- N candidate tactics $\{a_1, \ldots, a_N\}$
- Prior $P(s, a_i)$ = seq-level log-likelihood
- Value head = **expected remaining proof cost**

3 B
parameters

N ≈ 32
tactics / state

The Lean kernel is the filter

Tactics that don't parse are *silently dropped* — constraint enforcement is delegated to Lean. The policy is free to propose garbage, and that is *why* scale pays off here: even a 10% hit rate drives progress.

Why the value head matters

- It estimates remaining proof steps, not just success probability
- During training it learns where search is wasting depth
- During TTRL it is what makes search become *cheaper* — Fig. 3c

PUCT tree search — AlphaZero adapted to Lean

$$\mathrm{PUCT}(s,a) \;=\; Q(s,a) \;+\; c(s)\,\pi(a \mid s)^{1/\tau} \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}$$

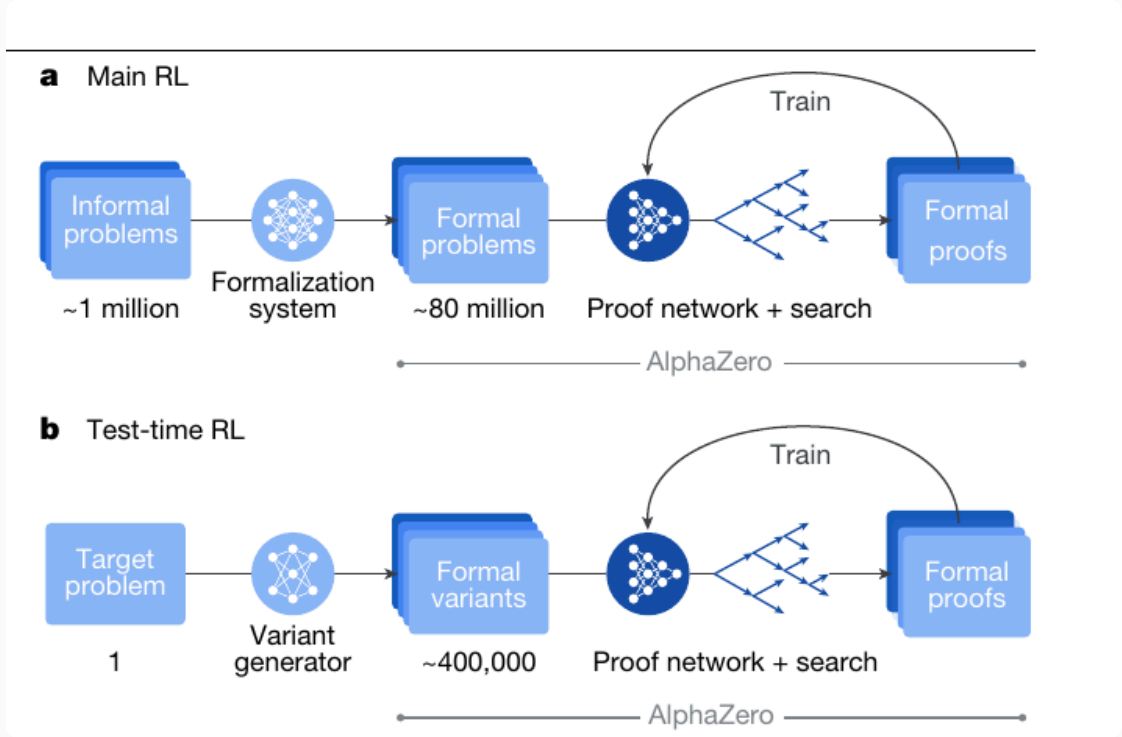
- Expand the leaf with the highest PUCT score
- Ask the proof network for N new tactics + V
- Apply each tactic in Lean → new child states
- Bootstrap values back up the tree
- In AlphaProof, $Q(s,a) = \gamma^{-V(s,a)-1}$
- Multi-subgoal nodes (⊗): value aggregated by **min**
- AND nodes prioritize the hardest unresolved subgoal
- The proof is returned by concatenating tactics along the successful branch
- Cost is measured in **simulations** per problem — one expansion + one proof-net call

PART III — TRAINING AT SCALE

Fabricate 80 million Lean statements, then teach yourself to prove them.

Auto-formalization from ~1 M informal seeds · AlphaZero-style actor/learner RL loop · match-maker curriculum.

Main RL training pipeline



~1 M
informal seed problems

~80 M
auto-formalized Lean statements

AlphaZero-style loop

- Actor hosts run PUCT searches and push trajectories to a replay buffer
- Learner updates the policy and value head from actor traces
- **Match maker** keeps problems near the agent's frontier of ability
- learner batches mix **10% Mathlib SFT** + **90% replay**

Main-RL compute: **~80 000 TPU-v6e days**

Auto-formalization — LaTeX in, Lean out

IMO 2021 Shortlist, Problem A5

Let $n \geq 2$ be an integer and let $a_1, a_2, \ldots, a_n$ be positive real numbers with sum 1. Prove that

$$\sum_{k=1}^{n} \frac{a_k}{1-a_k}(a_1 + a_2 + \cdots + a_{k-1})^2 < \frac{1}{3}.$$

Formalization system

```
theorem imo_shortlist_2021_a5
    (n : ℕ) (h₀ : 2 ≤ n) (a : ℕ → ℝ) (hapos : ∀ i, 0 < a i)
    (hasum : ∑ i in Finset.Icc 1 n, a i = 1) :
    ∑ k in Finset.Icc 1 n, a k / (1 - a k) * (∑ i in Finset.Icc 1 (k-1), a i) ^ 2 < 1 / 3
```

Accuracy on 50 held-out problems

	Alg.	NT	Comb.	Geom.	All	Pass@k
IMO	81.3	76.9	33.3	—	60.0	**96% @29**
Putnam	61.9	72.7	60.0	54.6	64.0	**98% @16**

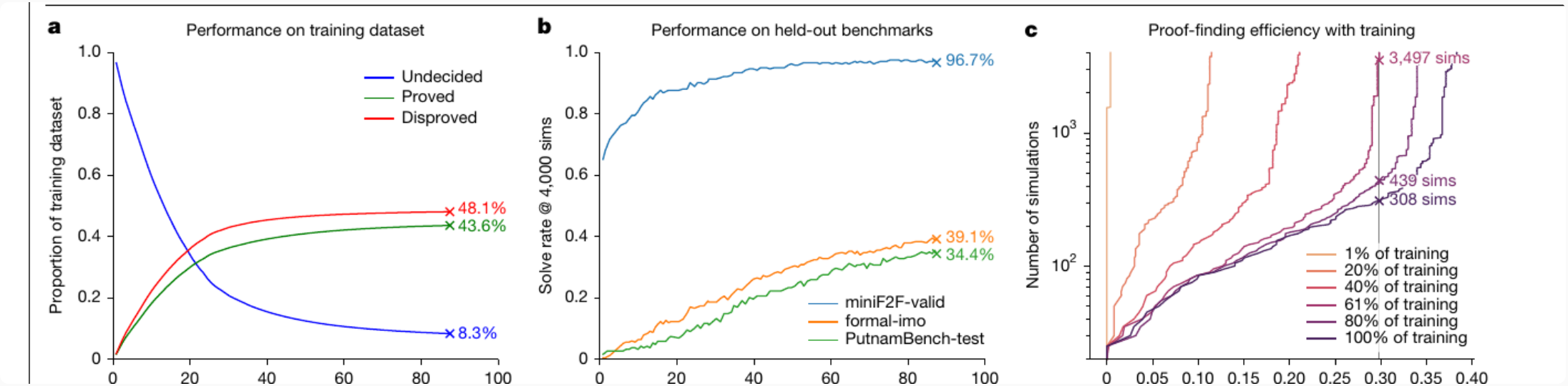
- Combinatorics lags — Mathlib gap, not model gap
- Geometry excluded from IMO — handed to AlphaGeometry 2
- At pass@k, **96–98%** of olympiad-level statements can be formalised correctly

Main RL — the actual control loop

```
for step in range(1_000_000):
    problem, sim_budget = matchmaker.sample_task()
    outcome = tree_search(problem, proof_network, sim_budget)
    matchmaker.update(problem, outcome)
    if outcome.is_terminal():
        replay.add(outcome.verified_trace)
    batch = mix_batches(mathlib_sft=0.10, replay=0.90)
    proof_network = learner_update(proof_network, batch)
```

- pretraining: **~300 B** public code/math tokens
- encoder processes **~12 T** tokens, decoder reconstructs **~3 T**
- SFT: **~300 000** Mathlib state-tactic pairs
- auto-formalization over project: **~100 000 TPU-days**
- main RL: **~80 000 TPU-days**
- stronger network + cheaper search is the whole point of the loop

Learning progression during main RL



(a) Training-set solve / disprove rate vs TPU-days · **(b)** Held-out benchmark climb · **(c)** Simulations-per-solve drops by **>1 order of magnitude** — search becomes cheaper as the network sharpens.

PART IV — THE KEY NEW IDEA

Test-time reinforcement learning.

The one component of AlphaProof with essentially no open-source analogue — and the single mechanism responsible for the silver medal.

Test-time RL (TTRL) in one sentence

> For each new hard problem, AlphaProof **generates a large curriculum of variants**, **runs RL** on those variants to fine-tune the proof network, **then** attacks the original target with the adapted weights.

Test-time compute → weight updates, not more samples.

- Same loss as main RL (CE on priors, MSE on values)
- Adapted weights are *discarded* after the target closes
- Variant generator: **Gemini-family LLM**

Variant curriculum details

- **Simplification** — weaken / restrict
- **Lemma** — propose a helper claim
- **Proof step** — assert an intermediate fact
- **791** few-shot `(problem, variant)` Lean pairs
- evolutionary refinement with $N_{\text{evo}} = 15$

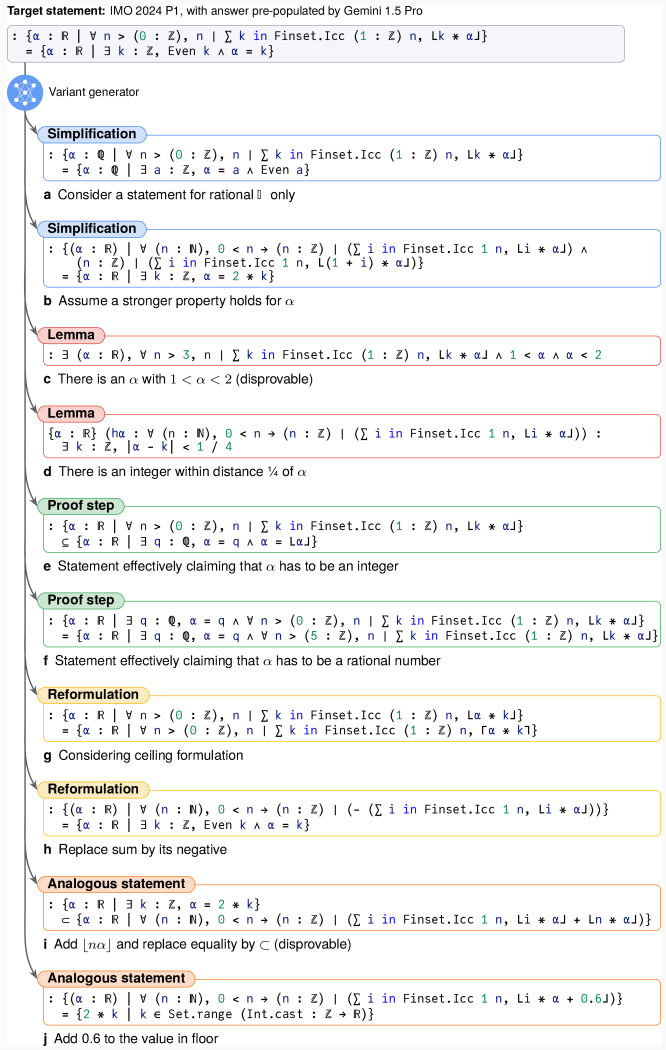
TTRL variants — IMO 2024 P1 in action

What the variant generator produces

- Provably true and provably false variants *both* help — disproof is just proving the negation.
- Labels on the right: green = proved, red = disprovable, yellow = reformulation.
- The generator uses **simplification**, **lemma proposal**, **proof step**, **reformulation**, and **analogous** moves.
- Takeaway: variants do not need to be true, they just need to be *close*.

The ablation punchline

- Going from 10 → 100 000 variants = **+5 pts** on the TTRL target set.
- A *harder* curriculum from a stronger LLM helps more than a cheaper one.



TTRL — the local learning loop

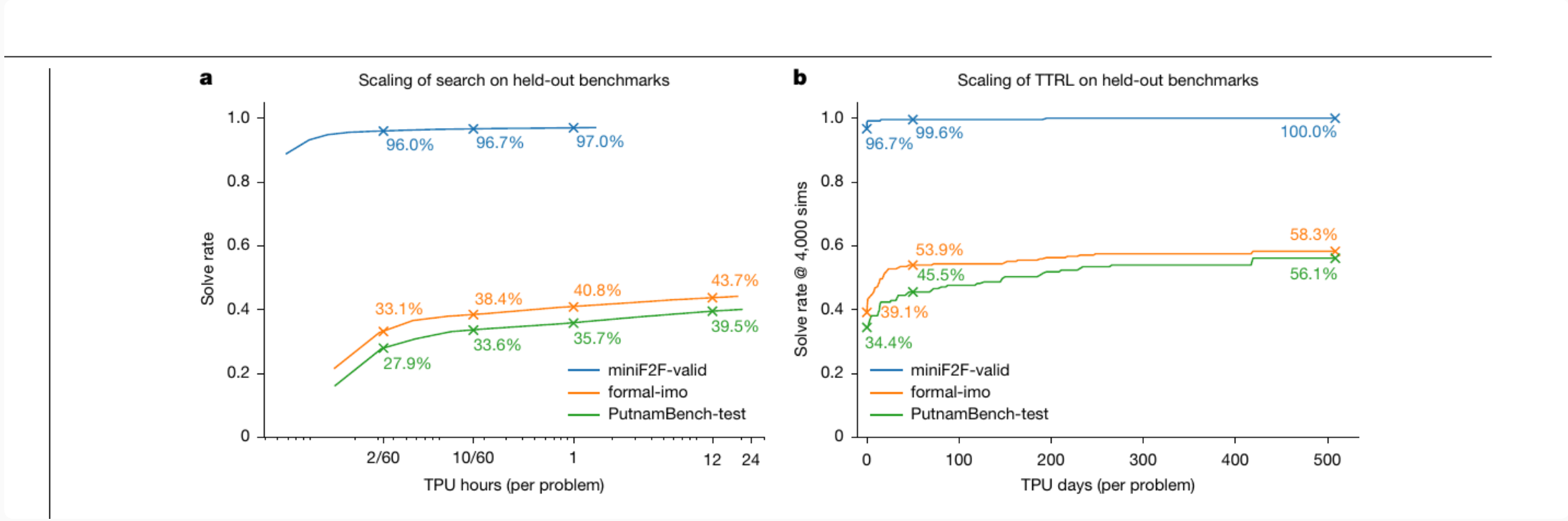
```
def ttrl(target, base, budget):
    variants = generate_variants(target)
    local = copy(base)
    replay = ReplayBuffer()

    for step in range(budget):
        v = sample_interesting_variant(variants)
        out = tree_search(v, local, sim_budget=4000)
        if out.is_terminal():
            replay.add(out.verified_trace)
        local = learner_update(local, replay.sample())

    return tree_search(target, local, sim_budget=4000)
```

- TTRL is **real SGD at test time**, not just "sample more"
- adapted weights are local and disposable
- reported solved IMO problems took **2–3 days of TTRL**
- this is the main capability gap between AlphaProof and open systems

TTRL cost curve — inference scaling



(a) Tree-search scaling — up to **12 TPU-h / problem**. **(b)** TTRL scaling — up to **500 TPU-days / problem**. On `formal-imo`: no TTRL → **33.2%**, full TTRL → **58.3%**. That 25-point gap is the silver-medal lift.

PART V — RESULTS

Benchmarks, scaling, and the 28/42 medal.

Headline numbers

System	miniF2F-test	formal-imo	PutnamBench-test
GPT-f expert iteration (2020)	36.6	—	—
HyperTree Proof Search (2022)	41.0	—	—
Kimina-Prover-Preview (2025)	80.7	—	1.6
DeepSeek-Prover-V2 (2025)	88.9	—	5.3
AlphaProof — 12 TPU-h	**97.7**	43.7	39.4
AlphaProof + TTRL — 500 TPU-days	**99.6**	**58.3**	**56.1**

miniF2F is saturated — top of the column is within 2 points of perfect. The real differentiator lives in `formal-imo` and `PutnamBench-test`, where TTRL buys AlphaProof a **+25 point** lead.

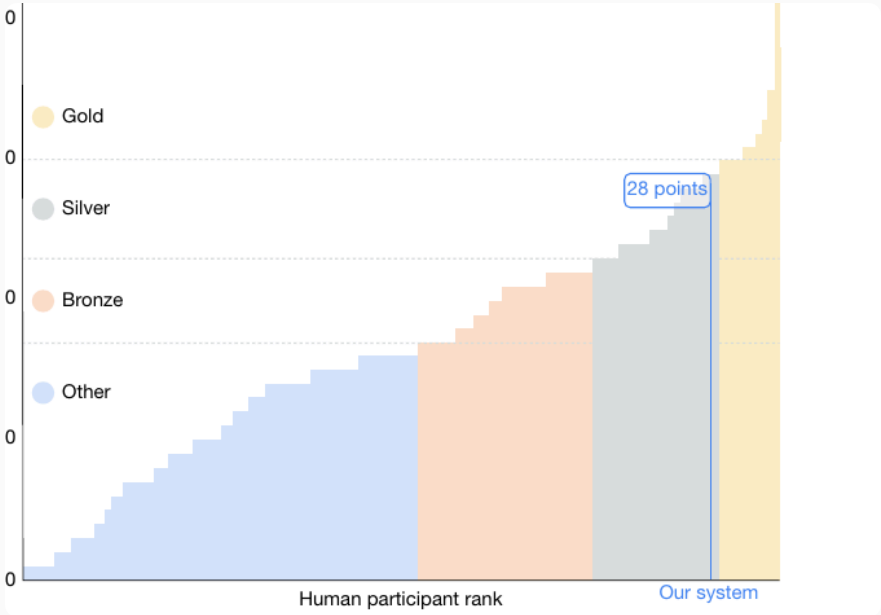
IMO 2024 — silver medal, one point from gold

Combined system score: 28 / 42

- **AlphaProof** solved **P1, P2, P6** (non-geometry)
- **AlphaGeometry 2** solved **P4** (geometry)
- **Gemini 1.5 Pro** guessed answer sets for parametric problems
- **P3, P5** — unsolved by any subsystem
- Medal thresholds: bronze 16 · silver 22 · gold 29
- solved AlphaProof IMO problems each took **~2–3 days of TTRL**

Honest caveats

- Wall-clock: *days* per problem, not the 9 hours of the real IMO
- P3/P5 were simply skipped — the split is by subsystem coverage, not strategy



The hardest problem — IMO 2024 P6

"Aquaesulian functions"

Fully solved by only **5 of 609** student contestants.

AlphaProof's key insight — highlighted line in the Lean proof — constructs the witness

$$f(x) = -x + 2\lceil x \rceil$$

to show the bound is tight.

The Lean proof is ~60 lines of `nlinarith`, `omega`, `norm_num`, and `simp_all` — correct but *not* a human-style proof. Automatically translating it to a readable LaTeX proof is a live research problem.

```
import Mathlib

set_option maxHeartbeats 2000000
open Polynomial
/--
Let ℚ be the set of rational numbers. A function f : ℚ → ℚ is called aquaesulian if the following property holds: for every x, y ∈ ℚ,

f(x + f(y)) = f(x) + y   or   f(f(x) + y) = x + f(y).

Show that there exists an integer c such that for any aquaesulian function f there are at most c different rational numbers of the form f(r)+f(−r)
for some rational number r.
-/
theorem imo_2024_p6
    (IsAquaesulian : (ℚ → ℚ) → Prop)
    (IsAquaesulian_def : ∀ f, IsAquaesulian f ↔ ∀ x y, f (x + f y) = f x + y ∨ f (f x + y) = x + f y) :
    IsLeast {c : ℤ | ∀ f, IsAquaesulian f → {f r + f (-r) | (r : ℚ)}.Finite ∧ {f r + f (-r) | (r : ℚ)}.ncard ≤ c} 2 := by
  exists@?_
  · useλu b=>if j:u 0=0then by_contra λc=>?_ else ?_
    · suffices:({J|∃k,u k+u (-k)= J}) ⊆{0}
      · simp_all[this.antisymm]
      rintro - ⟨a, rfl⟩
      contrapose! c
      simp_all
      suffices:{U|∃examples6, (u) ‹ℚ› +u ( -‹_›)= U} ⊆{0,(u (a : Rat)+ (u<|@@↑(( (-a )))) } ..
      · use ( Set.toFinite ( _) ).subset ↑@@this , (Set.ncard_le_ncard$ (((this )) ) ).trans (Set.ncard_pair$ Ne.symm (↑
        ↪ ( (c)) ) ).le
      rintro-⟨hz, rfl⟩
      induction b @hz a
      · have:=b (-a)$ hz+u a
        have:=b hz hz
        simp_all[add_comm]
        have:=b (-hz) (hz+u ↑(hz))
        simp_all[ add_assoc, C]
        induction this
        · simp_all
          have:=b hz (hz+(u a+u (-a)))
          have:=b (hz+(u a+u (-a)))$ hz+(u a+u (-a))
          use .inr$ by_contra$ by hint
        have:=b hz$ hz+(u hz+u (-hz))
        cases b (hz+(u hz+u (-hz)))$ hz+(u hz+u (-hz))with|_=>hint
      have:=b (-hz) (u hz+a)
      have:=b$ -a
      specialize this (u hz+a)
      simp_all[ ←add_assoc]
      have:=b 0
      have:=b
      specialize b a a
      simp_all[add_comm]
      have:=(this<| -a) (↑a + (((u a))): (↑_ :((( _) ) ) )) ..
      simp_all[add_assoc]
      cases this
      · simp_all
        contrapose! IsAquaesulian_def
        simp_all
        exfalso
        have:=this a (a+(u hz+u ( -hz)))
        simp_all[Ne.symm,Bool]
        have:=‹∀congr_arg G,_› (a+(u hz+u (-hz)))$ a+(u ↑hz+u ↑( -hz) )
        simp_all
      have:=this a (a +(u a+u (-a)))
      cases‹forall Jd S,_› (a+(u a+u (-a))) ( a + (u a +u ↑(-a)))with| _ =>hint
    simp_all
    cases b 0 0with|_=>exact absurd (b 0$ (0+(1 *(@(u ↑.((0) )))))^ 01: ↑ ((_)) ) (id$ (by(cases ( b (u 0) ( (u
      ↪ 0)))with|_ => continuity)))
  rintro K V
  specialize V $ λ N=>-N+2 *Int.ceil N
  specialize( V $ (IsAquaesulian_def _).mpr _)
  · simp_rw [ ←eq_sub_iff_add_eq']
    ring
    use mod_cast@?_
    norm_num[<-add_mul,Int.ceil_eq_iff]
    useλc K=>(em _).imp (⟨by linarith[Int.ceil_lt_add_one c,Int.le_ceil K],.⟩) (by repeat use by linarith[.,Int.le_ceil
      ↪ c,or,Int.ceil_lt_add_one$ K])
  simp_all[Int.ceil_neg, ←add_assoc]
  suffices:2<=V.1.toFinset.card
  · let M:=V.1.toFinset
    norm_num[this,V.2.trans',(Set.ext$ by simp_all[M]: {x :Rat|∃t:Rat, (↑2 ) * ( ⌈ t ⌉:(ℚ ) ) .. + (- (2 *L(t)⌋)) = ↑x} =
      ↪ M)]
  use Finset.one_lt_card.2$ by exists@0,V.1.mem_toFinset.2 (by exists-1),2,V.1.mem_toFinset.2 (by exists 1/2)
```

PART VI — THE OPEN ECOSYSTEM

What can an academic lab run next week?

The open ecosystem

System	miniF2F-test	Why it matters
BFS-Prover-V2-32B	**95.08**	strongest compact open result
DeepSeek-Prover-V2	88.9	strongest subgoal-decomposition RL story
Goedel-Prover-V2-32B	88.1 / 90.4	Apache-2.0, strong 8B line
LeanDojo + ReProver	~26	the main open substrate
Lean Copilot	n/a	easiest in-editor assistant

How to read the landscape

- `BFS-Prover-V2-32B` is the best open "just run the prover" baseline here.
- `LeanDojo + ReProver` is the best starting point if you want to build a lab stack.
- `DeepSeek-Prover-V2` and `Goedel` are the most relevant open comparisons for RL-flavoured proving.
- `Kimina-Prover-72B` is the closest open analogue of TTRL-style adaptation, but its headline score is from a blog post, not peer-reviewed.

A Lean 4 example — what the network has to learn

```
-- a² + b² ≥ 2 a b    (AM-GM, one-line proof)
example (a b : ℝ) : a^2 + b^2 ≥ 2 * a * b := by
  nlinarith [sq_nonneg (a - b)]

-- x + y = 1   ⇒   x · y ≤ 1/4
example (x y : ℝ) (h : x + y = 1) :
    x * y ≤ 1 / 4 := by
  nlinarith [sq_nonneg (x - y),
             sq_nonneg (x + y - 1)]
```

What the policy actually learns

- Not the *theorem* — Lean's `nlinarith` finishes both of these.
- The skill is choosing **which `sq_nonneg` hints to pass**.
- Search space: all finite subsets of Mathlib's ~200 k lemmas.
- The proof-network value head pushes it toward the ~3 hints that actually work.

> *Tactic selection, not theorem proving, is what the RL loop optimises.*

End-to-end on one theorem

```
theorem odd_sq_sub_one_dvd_eight (n : Nat) (h : Odd n) : 8 ∣ n ^ 2 - 1 := by
  rcases h with ⟨m, rfl⟩
  have eq : (2 * m + 1) ^ 2 - 1 = 4 * (m * (m + 1)) := by
    ring_nf
    grind
  simpa [eq] using
    (mul_dvd_mul_iff_left four_ne_zero).mpr (Nat.two_dvd_mul_add_one m)
```

- informal theorem
- auto-formalize to Lean
- open tactic state
- tree search over tactics
- Lean verifies the finished proof term
- outer term skeleton is still just `match`, `let`, transport, and library lemmas
- this is the smallest full AlphaProof story you can tell on one slide

The local algebra certificate

Human mathematical content

$$
\begin{aligned}
(2m+1)^2 - 1 &= (4m^2 + 4m + 1) - 1 \\
&= 4m^2 + 4m \\
&= 4(m^2 + m) \\
&= 4m(m+1)
\end{aligned}
$$

$$eq = F_m \circ F_4 \circ C \circ N$$

- `N` : normalize and expand
- `C` : cancel `+1 - 1`
- `F_4` : factor out `4`
- `F_m` : factor out `m`

What Lean actually checks

```
theorem ... _proof_1_3 :
    1 + m * 4 + m ^ 2 * 4 - 1
      = m * 4 + m ^ 2 * 4 :=
fun m =>
  Classical.byContradiction
    (Lean.Grind.intro_with_eq ...)
```


- `ring_nf` builds a large congruence tree

PART VII — REFLECTION

What is new, what is expensive, what is next.

Honest compute accounting

Configuration	Per problem	What it buys
AlphaProof — fast	**2 TPU-min**	96% miniF2F
AlphaProof — strong	**12 TPU-h**	97.7% miniF2F · 43.7% formal-imo
AlphaProof + TTRL — peak	**500 TPU-days**	**99.6% · 58.3% · 56.1%**

- **Main RL** ≈ 80 000 TPU-v6e days ≈ 4 000 TPUs × 20 days
- **Auto-formalization** ≈ 100 000 TPU-days over the project
- **Solved IMO 2024 problems** took roughly 2–3 days of TTRL each

The paper reports TPU budgets, not a vendor-normalized dollar cost. The sourced point is compute scale, not a precise invoice.

What is *actually new* in AlphaProof?

Already in the literature

- AlphaZero policy/value + search
- GPT-f / PACT tactic models
- HTPS-style MCTS on proofs
- LLM auto-formalization
- Expert iteration + match-making

Every block above has an open-source parallel.

Genuinely new

- **TTRL** — per-problem reinforcement learning at test time
- No direct open-source analogue (Kimina-72B is the closest)
- Responsible for the **+25 pt** lift on `formal-imo`
- The single algorithmic contribution worth stealing

Limits and honest caveats

1. **Combinatorics is a wall.** Both the formalizer and the solver stall — `formal-imo` solve rate plateaus at **~15%** even with full TTRL.
2. **Mathlib gaps bound the ceiling.** Some problems cannot even be *formalised* because the underlying theory is missing from the library.
3. **Geometry is outsourced.** P4 at IMO 2024 is AlphaGeometry 2, not AlphaProof.
4. **"Find all X" problems need a guesser.** Answer sets come from Gemini 1.5 Pro *before* AlphaProof starts proving.
5. **Compute.** See the previous slide.

Three ideas worth stealing

1. Verifier-grounded RL works

The Lean kernel is a perfect, sparse reward model. **No preference data, no reward-model, no human labels.**

2. Auto-formalization is a force multiplier

Formal data scarcity is an LLM *translation* problem, not a hard wall. ~80 M statements from ~1 M informal seeds.

3. Test-time RL changes the cost curve

Spend inference compute on *gradient updates on problem-specific variants*, not on more samples. A new axis of capability.

Where next?

- **Make TTRL cheap.** Smarter variant generation + scheduling. Probably the highest-leverage research direction in the field right now.
- **Generalise the verifier.** Isabelle, Coq, Agda, and domain-specific provers (probability, PDEs, …) all work with the same MDP framing.
- **Real interactive collaboration.** AlphaProof is a *batch* system; Lean Copilot is *interactive* — close the gap.
- **Close the combinatorics gap.** Tactic design **and** Mathlib formalisation work both needed.
- **Beyond olympiad.** Research-level mathematics as the next benchmark — the competition era is over.

THANK YOU

Questions?

Paper — Hubert *et al.*, *Nature* **651**, 607–613 (2026) · DOI `10.1038/s41586-025-09833-y`

Lean 4 — `leanprover.github.io` · **LeanDojo** — `leandojo.org`

DeepSeek-Prover-V2 · **Kimina-Prover** · **Goedel-Prover-V2** · **Lean Copilot**

Companion — `jupyterbook/11_alphaproof_linear_walkthrough.md` · `jupyterbook/notes/alphaproof_linear_walkthrough.pdf`